# Attributing Subagent Work in Coding-Agent Protocol Adapters

Rajasekhar Josyula
Independent Researcher
rajasekhar.josyula@gmail.com

Lakshmi Sravya Vedantham
Independent Researcher
lakshmisravya.vedantham@gmail.com

***Abstract*. Editors reach coding agents through Agent Client Protocol (ACP) adapters, some of which also tell the editor which files each request changed. When a subagent outlives the request that started it, its late edits can be credited to whichever request is current when they arrive. We state the property such reports should keep, an origin-attribution invariant, and check it with TURNSTILE, which runs an unmodified adapter against a scripted provider under the event orders a provider model allows in hand-written scenarios, exhaustively or, for four large ones, within a preemption bound. The Codex model follows provider source; the Claude model, the adapter's own assumptions. In the open-source Codex and Claude ACP adapters, an edit made for request A appears in B's report in 581 of 956 Codex schedules and 39 of 111 Claude schedules, and in single live runs of both CLIs. Binding each subagent to the request current when it starts removes the 39 Claude failures but not a live one, and leaves 174 Codex ones; binding Codex subagents to the turn that dispatched them removes every misattribution in these scenarios but omits some of B's own subagent work. We prove that on the typed Codex 0.160.1 wire no adapter meeting stated conditions attributes exactly in all legal executions, and that origin links on the event stream suffice. A source screen of eight further ACP implementations found no other instance.**

***Index Terms*. LLM agents, agent protocols, multi-agent systems, provenance, concurrency testing, schedule exploration**

## I. INTRODUCTION

Editors drive coding agents through protocol adapters. The Agent Client Protocol (ACP) [1] lets an IDE talk to any compliant agent, and adapters wrap vendor runtimes (*providers*) such as Codex and Claude Code. Some adapters also derive per-request outputs, such as the files one request changed; the Codex and Claude adapters do so for JetBrains AIR [2].

Agents now delegate to subagents, and some providers let a subagent outlive the turn that started it. Both adapters we study keep per-request bookkeeping, created for each request and disposed when it returns, so a late event from an older subagent can arrive after the state that recorded its origin is gone (Fig. 1). Credited to the request active on arrival, it puts one request's changes in another's report, and the client cannot tell: the report has the right request id and the wrong contents.

Upstream file-report tests never send a second request while a subagent of the first still runs, and a hand-picked failing order invites the objection that providers never produce it, so we enumerate the orders a provider model allows. We contribute:

1) an **origin-attribution invariant** for derived per-request outputs, with proofs that, on the typed Codex 0.160.1 wire, adapters meeting stated conditions cannot always attribute exactly, and that origin links on the event stream suffice (Sec. III);
2) **TURNSTILE**, an LLM-free checker in the style of systematic concurrency testing [3] that runs the unmodified adapter under the schedules a provider model allows and checks each report against a ledger of origins (Sec. IV);
3) **findings**: one defect class in the Codex and Claude ACP adapters, found by source review, reproduced live and by TURNSTILE; two fixes, evaluated; and a source screen of eight further implementations (Secs. V and VI).

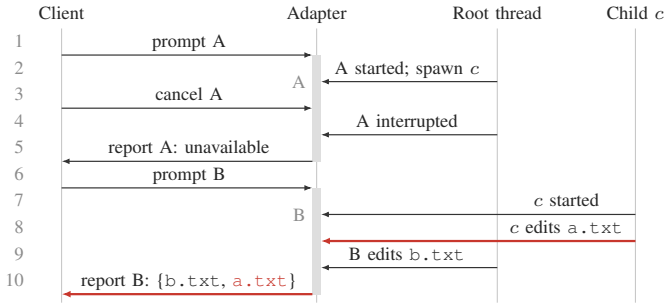


Fig. 1. Minimal failing cancel schedule (abridged; no preemption). The original adapter (step 8) and the start-time patch (step 7) credit $c$'s edit to B; the dispatch-time patch binds $c$ to A via the spawn item (step 2).

## II. BACKGROUND

**ACP** is a JSON-RPC protocol, usually over stdio, between a client and a coding agent (not MCP, A2A, or the Agent Communication Protocol).

**Prompt turns.** A client sends `session/prompt`; the agent streams `session/updates` and answers with a stop reason. After `session/cancel` the agent may still send updates, but only before it answers [1]. ACP's subagent proposal lets "a background child that the cancelled work did not wait on" continue and scopes its output to a child session [4], so a child event during a later request is not by itself a protocol violation. A *request* is one `session/prompt` call (A, B, ...); a *turn* is provider-side work: one *root turn* per request, plus *child turns* run by subagents (*children*).

**Derived reports.** In the AIR extension [2] the client asks for an *agent file-change report* with a fresh `requestId`, and the adapter returns the paths changed by that request before answering it; a `declaredComplete` flag can admit

omissions. The documentation does not say who owns paths changed by children.

## III. Origin Attribution: Invariant and Limits

Each turn has an *origin*: the request whose processing dispatched it, inherited by nested children. Informally, a report for $r$ should list only paths of origin-$r$ work, and all of them seen so far or say it may be incomplete. A late write by A's child may be legitimate; listing it as B's is not. Violations are *turn-attribution bugs*; the usual one, *arrival-time attribution*, takes the request active *now* as the origin. The invariant is our proposal, not part of the AIR contract.

**Model.** Requests $r = A, B, \ldots$ arrive in order. A child turn $w$ is started by one *dispatch* (a spawn or resume, or Codex *trigger* mail, P1) from turn $\delta(w)$ (on Codex, the recorded parent); queue-only mail, mail taken by a running turn, and interrupts are not dispatches, and executions with a child turn lacking a unique dispatcher are excluded. Thread streams are FIFO, mutually unordered, unboundedly delayed; *legal* executions obey the provider. A deterministic adapter maps its input (messages and arrival times) to provider requests and, after $\rho_r$ ends, one report $(R_r, c_r)$, *available* unless the client cancels $r$ or $\rho_r$ is interrupted without the adapter's request.

| | | | |
|---|---|---|---|
| $\rho_r$ | root turn of $r$ | $\delta(w)$ | sender of $w$'s dispatch |
| $o(w)$ | $o(\rho_r)=r$, $o(w)=o(\delta(w))$ | $R_r$, $c_r$ | report for $r$, its flag |
| $W_r(x)$ | turns whose $x$ came before $R_r$ | $Own_r$ | $x$ with $o(w)=r$, $w \in W_r(x)$ |

*Definition 1:* An available report is *sound* (the *origin-attribution invariant*) if each $x \in R_r$ has a $w \in W_r(x)$ with $o(w) = r$, *complete* if $Own_r \subseteq R_r$, and *safe* if sound with $c_r$ false unless complete. An adapter is *exact* (*safe*) if, in all legal executions, its available reports are sound and complete (safe). Omission ($R_r = \emptyset$, $c_r$ false) is always safe.

The flag $c_r$ is `declaredComplete`. Both adapters always clear it, so for them safety is soundness.

**Codex 0.160.1** [5] premises (multi-agent V2, no hooks; each read in the source): (P1) `followup_task` and `send_message` emit the same item under the caller's turn; only the former's mail is a trigger, with the caller's turn as parent and its settings. (P2) Mail to a busy turn waits, starting nothing, until its next model call. (P3) An interrupt aborts only its target's turn; after `interrupt_agent`, if trigger mail waits, a new turn with a random id takes all mail, the triggers' common parent and the last one's settings. (P4) Except P1's and P5's items, no typed (non-raw) notification, reply to a turn, goal or thread-opening request, history read (`thread/read`, `list`, `turns/list`, `items/list`), or background-terminal list shows mail, its tool, trigger flags, or parent or root ids. (P5) An interrupted child turn sends no completion mail or item (naming its parent). (P6, modeling) Model API replies (including token usage), all timing, and random ids are free. The adapter (Q1) after a client cancel during $\rho_r$ answers $r$ and forwards the next prompt without waiting for $r$'s children and, for a client that only prompts and cancels, (Q2) runs all requests on one live hook-free thread, lets V2 agent tools and child edits run unasked, and acts on children only by reads and replies, and (Q3) takes all typed notifications, and thread data only from P4's sources (raw events show mail types). By our source trace of every Codex request and file read, codex-acp's default mode meets Q1–Q3 (Q1 if it interrupts $\rho_r$ in time, P6).

| | Same in $E$ and $E'$ | Hidden in $E$ | Hidden in $E'$ |
|---|---|---|---|
| 1 | $\rho_A$ spawns $c$; $c$'s turn $v$ runs | | |
| 2 | $\rho_A$ mails $c$ | task | plain message |
| 3 | client cancels A; $\rho_A$ ends | | |
| 4 | B's prompt is forwarded; $\rho_B$ starts | | |
| 5 | $\rho_B$ mails $c$ | plain message | task |
| 6 | $\rho_B$ interrupts $c$; $v$ ends | | |
| 7 | new turn $u$ takes both mails, edits $x$ | parent $\rho_A$ | parent $\rho_B$ |
| 8 | $\rho_B$ interrupts $u$; $\rho_B$ ends; report due | | |
| | origin of $u$, so owner of $x$ | A | B |

Fig. 2. The witness pair of Theorem 1: the same adapter input, but which mail is a task differs, and with it $u$'s parent and so the owner of $x$.

**Witness.** In $E$, $\rho_A$ spawns $c$, whose turn $v$ runs a long tool call, and sends $c$ a `followup_task`. The client cancels A during $\rho_A$; once $\rho_A$ ends, B is forwarded. $\rho_B$ sends $c$ a `send_message` and interrupts $v$; a new turn $u$ takes both mails with parent $\rho_A$ (P2, P3), so $o(u) = A$. Only $u$ edits $x$; the diff arrives before $\rho_B$ interrupts $u$ and ends without edits. $E'$ swaps the two tools, so $o(u) = B$ (Fig. 2).

*Lemma 1:* Under P1–P6 and Q1–Q3 in $E$, $E$ and $E'$ are legal and give the adapter identical input.

*Proof:* Each step is legal (P1–P3, Q2, Q3) whatever else the adapter does: B is forwarded while $v$ runs (Q1, P3); the adapter acts on $c$ only by reads and replies (Q2), and any that would end $v$ or $\rho_B$ early is processed after $u$'s interrupt (P6). While inputs agree, so do the adapter's requests. Only tool names, the mails' trigger flags and headers, and $u$'s parent, root and inherited settings differ. No input shows them (P4, Q3); the settings reach only the model API (u calls no MCP tool), whose replies are chosen equal (P6). All ids, times, texts and token counts are chosen equal (P1, P3, P6), and no completion is ever sent for $v$ or $u$ (P5), so waiting reveals nothing. ∎

*Theorem 1 (No exact attribution on the typed Codex 0.160.1 wire):* Under P1–P6, no adapter meeting Q1–Q3 in $E$ is exact (origins by $\delta$); a safe one clears $c_B$ in $E$, where B's report is complete.

*Proof:* Only the adapter may end B early, so by Lemma 1 it publishes one $(R_B, c_B)$ for both runs. If $x \in R_B$, the report is unsound in $E$ ($W_B(x) = \{u\}$, $o(u) = A$). Otherwise $x \in Own_B \setminus R_B$ in $E'$, and safety forces $c_B$ false in $E'$ and so in $E$, where $Own_B = \emptyset$. ∎

The proof is a two-execution indistinguishability argument [6]. Turnstile renders the pair as one message sequence with two ledgers (Table I, pair rows), so the run shows how each build answers this ambiguity, not that both executions occur on real Codex; that rests on P1–P6. Every build answers both runs alike. In the 792 of 952 schedules where $u$'s diff precedes B's report, the original and start-time builds list $x$ in $R_B$, a misattribution in $E$, and our dispatch-time patch (Sec. V), which also meets Q1–Q3, omits $x$ with a warning, missing B's edit in $E'$. No other scenario starts a child turn on two requests' mail, so Theorem 1 explains no other failure.

**Sufficiency.** A lineage rule $\lambda$ maps each $m \in M_r$, the *evidence* (edit messages) that arrived before $R_r$, to a request or $\perp$, using messages before $R_r$. It is *truthful* (*timely*) if in all legal executions $\lambda(m) = r'$ implies $o(w) = r'$ for $m$'s sender $w$ ($\lambda(m) \neq \perp$). Rule $T_\lambda$ reports $x$ for $r$ iff some evidence for $x$ in $M_r$ has $\lambda = r$, and sets $c_r$ iff no $\lambda(m)$ is $\perp$.

*Proposition 1 (Lineage suffices):* If $\lambda$ is truthful, Rule $T_\lambda$ is safe; if $\lambda$ is also timely, it is exact.

*Proof:* Truthfulness gives each $x \in R_r$ a sender of origin $r$. If $c_r$ is true, the evidence of an origin-$r$ sender of each $x \in Own_r$ resolved, to $r$, so $x \in R_r$. Timeliness sets $c_r$. ■

Codex 0.162.0 adds `rootTurnId` to turns and their lifecycle notifications [7], which separates $E$ from $E'$. By our source reading, consistent with single live runs, mapping $m$ to the request whose root turn is the `rootTurnId` on the `turn/started` of $m$'s turn is truthful and timely, and so makes $T_\lambda$ exact (Prop. 1, applied per execution), in V2 executions where turn ids are distinct, each request's root turn is its own root, every report follows all `turn/start` replies and precedes the next prompt's forwarding, and every other turn on an evidence sender's dispatch chain was started by spawn or follow-up mail from one turn. Neither the pinned nor the current codex-acp reads that field. For subagent edits our Claude patch is $T_\lambda$ with $c_r$ never set, where $\lambda$ is the request active when a subagent's launch or resume was registered (else at arrival); it misattributes, e.g., subagents launched or resumed by earlier work during a later request.

## IV. Turnstile: Exploring Protocol-Legal Schedules

**Provider substitution.** Both adapters start their provider as a child process whose path an environment variable overrides; Turnstile points it at a relay that forwards every line to the explorer. The adapter is built unmodified from the pinned commit. A *provider stub* per wire protocol (Codex app-server JSON-RPC, Claude CLI stream-json) renders abstract scenario events (*root turn started*, *spawn child*, *child edit*, ...) into concrete messages; a minimal ACP client sends prompts and cancels and captures reports.

**Protocol-legal schedules.** A scenario is a set of items (provider events and client actions), each on a FIFO *stream* (one per provider thread, plus the client's), with optional happens-before edges and *triggers* that enable an item only after the adapter has sent a given message. For Codex the rules combine guarantees confirmed in the pinned provider's source with restrictions that only remove legal orders, so they can hide failures but not create illegal orders. Guarantees: events within a thread are FIFO; `turn/interrupt` reaches only the root thread, so children keep emitting after it; nothing orders events across threads; and a parent waits for children only through a wait tool. Restrictions: a child's events follow the root's spawn item, B's turn follows A's completion, a completion report follows its own completion, and a few similar causal orders. For Claude we could not check provider source; the stub encodes the adapter's own assumptions (background-subagent frames share the CLI's single ordered stream, are tagged with their launching tool call, and may interleave with later root turns), each seen in live Claude runs.

A *schedule* is one total order of a scenario's 10 to 24 items that respects these constraints. The explorer runs each schedule in a fresh adapter process, delivering one item at a time and waiting until neither side has sent a message for 150 ms, a heuristic for quiescence; it explores message orders, not races inside the adapter. A *preemption* is a switch away from a provider stream that still has a deliverable item [3]; it measures distance from the default order, not likelihood.

**Oracle.** Each scenario carries a ledger, hidden from the adapter, that maps every evidence path to its true origin. The oracle reports a *misattribution* when a report lists a path of another request's origin, and *missing expected output* when a report omits a required own path whose evidence arrived before it. It also flags missing, unexpected, early, or late reports, false completeness claims, and adapter exits; an omission in a report marked incomplete is only recorded. A schedule that hangs, errors, or diverges on re-run is *inconclusive*, never clean. Violating schedules are re-run three times and count only if they fail identically. Four deliberately broken builds (no B report, dropped child paths, an early report, a crash) were each flagged on every affected schedule.

**Controls.** Fixes must not win by discarding work, so each adapter has controls in which late child output is legitimately B's (B starts a new child, gives A's child new work, or resumes it) or must stay out of B's report (an unregistered thread, the capability off, a late diff with no B). A correct adapter passes every control in every schedule.

## V. Results

We pinned codex-acp at commit `082be156` and claude-agent-acp at `ecaa2c93`; the Codex stub follows rust-v0.160.1 (`d27764b8`), the version codex-acp pins [5]. Codex runs require the client to advertise native subagent sessions. Table I comes from one final run of every build, with no divergent or inconclusive schedule and no flaky one except as noted. Stubs call no LLM; on one desktop each build's main Codex runs took under an hour. In the groups of Fig. 3, an outcome depends only on when the child's evidence arrives relative to B.

**Codex ACP.** The event handler assigns a child turn it has not seen to the *current* root turn. Each request builds a new handler whose disposal clears these bindings, but the session's child subscriptions survive and route into whichever request is current. We found no failure when A completes normally, as under V2 the adapter holds A open until the child's `turn/completed`, up to a 10-minute timeout [8].

**Claude ACP.** The adapter records child edits into the session's mutable `activeTurn`. When the CLI ends A's turn while A's background subagent runs, the adapter holds A open [9]; a B sent meanwhile becomes active when the CLI echoes it, and a subagent edit after that echo lands in B's report. We did not model cancellation.

**Start-time binding.** Our first fix, for both adapters, binds each child to the root turn current when the child's start arrives.

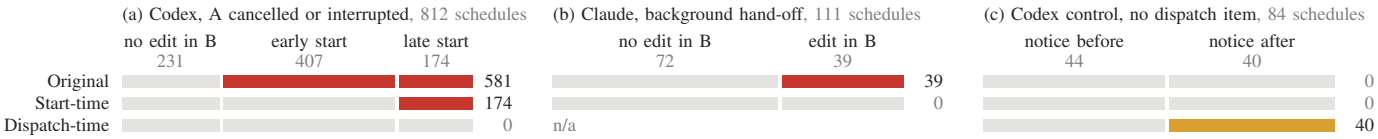


Fig. 3. Every schedule of three scenario groups, split (to scale) by when the child's evidence reaches the adapter; one bar per build (misattribution, missing output, clean), violating schedules counted at right. *In B*: after B's prompt (Codex) or its echo by the CLI (Claude), before B's report. *Early/late start*: in B, child started before/after B's prompt. *Notice before/after*: Codex's completion notice for the child's new turn came before/after B's report.

TABLE I
TURNSTILE RESULTS PER BUILD: SCHEDULES EXPLORED (EXHAUSTIVE EXCEPT ROW[c]) AND SCHEDULES IN WHICH A REPORT LISTS ANOTHER REQUEST'S EDIT OR OMITS AN OWN EDIT THAT ARRIVED BEFORE IT.

| | | Misattributing | | | Omitting own edit | | |
|---|---|---|---|---|---|---|---|
| Scenario | Sched. | Orig. | Start | Disp. | Orig. | Start | Disp. |
| *Codex* | | | | | | | |
| A cancelled by client | 504 | 341 | 87 | 0 | 0 | 0 | 0 |
| A interrupted[a] | 308 | 240 | 87 | 0 | 0 | 0 | 0 |
| A completed | 144 | 0 | 0 | 0 | 0 | 0 | 0 |
| sibling or grandchild[c] | 996 | 620 | 620 | 0 | 0 | 0 | 226 |
| 6 controls | 401 | 0 | 0 | 0 | 0 | 0 | 40[b] |
| Thm. 1 pair, $E$ | 952 | 792 | 792 | 0 | 0 | 0 | 0 |
| Thm. 1 pair, $E'$ | 952 | 0 | 0 | 0 | 0 | 0 | 792[b] |
| *Claude* | | | | | | | |
| background child hand-off | 111 | 39 | 0 | n/a | 0 | 0 | n/a |
| 2 controls | 30 | 0 | 0 | n/a | 0 | 0 | n/a |

Codex runs use multi-agent V2; the first three rows each cover three delivery modes (first diff, changed snapshot, identical replay). [a]Interrupt issued by the server or another client. [b]Required paths, a violation (missing expected output); other omissions are allowed, as both adapters mark every report possibly incomplete. [c]Four scenarios, every schedule without preemption; within one preemption Disp. misattributes in none of 22,074 and omits in 6,760. Misattributions reproduced identically on re-run; in row $E$ two Start ones each had a re-run stall under load and failed 3 of 3 when re-run alone. Patch SHA-256: Codex start `c36199f4`, dispatch `594c7ecf`; Claude `71e7b150`.

Both patches pass our reproduction tests and the upstream file-report tests (Codex 28/28, Claude 34/34). The fix removes every Claude violation, but A's child edits after A's report then go unreported. On Codex a child's `turn/started` can itself arrive after B's prompt, which binds the child to B (Fig. 1): binding at the start event is still an arrival-time proxy.

**Dispatch-time binding (Codex).** The second fix binds a child turn through the item that dispatched it: a spawn or follow-up item names the child's thread under the dispatching turn's id. Since the wire does not say which dispatch started which child turn, the patch pairs them in arrival order and grades each binding *named* (Codex's completion notice names the parent turn, or the turn is a spawned thread's first), *paired* (nothing contradicts the pairing), or *uncertain*; uncertain work is left out, and the report says so. The patch removes every misattribution in Table I, including where an A-era sibling or grandchild of A's child follows it up while B does, though it then omits B's own child edit in 226 of 996 schedules (start-time misattributes in 620). Those scenarios have millions of legal orders each, so we bound preemptions as CHESS does [3]. All builds pass a 254-schedule control in which B gives A's child work through a follow-up item. A control with *no* such dispatch item, although real Codex emits one, fails in 40 of 84 schedules (Fig. 3c): the only origin evidence is Codex's completion notice naming B's turn, and when it arrives after B's report the patch omits the edit and flags the report. This is missing expected output, not misattribution, in a scenario harsher than the behavior it abstracts; it does not show that the loss is unavoidable. The patched adapter passes its suite except our dispatch-less test (1,815 passed, 36 skipped, Codex-binary tests excluded).

TABLE II
SOURCE SCREEN OF ACP IMPLEMENTATIONS (PURPOSIVE SAMPLE). A GUARD MEANS ONLY THAT WE FOUND NO REACHABLE FAILING PATH.

| System | Attribution | System | Attribution (guard) |
|---|---|---|---|
| Codex ACP* | arrival-time | Pi ACP | arrival window (settle, drain) |
| Claude ACP* | arrival-time (child edits) | OpenCode | msg-id anchor (1 prompt, drain) |
| Muse ACP | explicit turn id | Goose | session snapshot (1 run, drain) |
| Gemini CLI | prompt-local | fast-agent | arrival window (lock, merge) |

Codex, Claude, and Muse report AIR file changes, Pi ACP usage and files, the others usage. Mistral Vibe reports only a session total, Cline no per-turn output. *Found by source review; reproduced by TURNSTILE (Sec. V).

**Live runs.** In single runs of the CLIs (Codex 0.160.1, 0.162.0; Claude Code 2.1.309) staged so that A's child edited during B, the original builds put that edit in B's report: Codex with A cancelled, Claude if B was sent before A's answer or the CLI resumed the subagent in B. Neither 0.160.1 fix did (an early start, Fig. 3a); the Claude patch did on the resume. On 0.162.0 each child turn's `rootTurnId` named its dispatching request; a patch reading it failed a cancel run with a per-prompt map and passed another with a per-session one.

## VI. SOURCE SCREEN OF ACP IMPLEMENTATIONS

Writing a provider stub is the main per-adapter cost, so we also read the event-to-report path of eight further open-source ACP implementations at pinned main-branch commits and classified how each attributes per-turn output (Table II).

Work outliving its request is necessary for the bug but not sufficient: OpenCode also lets background children survive a cancelled turn but hands them to a muted consumer that cannot write B's report. Guarded implementations carry origin explicitly (Muse), keep state unreachable across requests (Gemini CLI), or prevent arrival during B by rejecting or serializing prompts or draining the previous turn.

## VII. LIMITATIONS AND THREATS TO VALIDITY

**Scope and frequency.** Exploration is exhaustive only within hand-written scenarios shaped by defects that source review

had already found, and for four of them only within a preemption bound. We did not measure how often providers produce these orders (a live run shows one order) and found no open-source client that consumes the report, so we claim neither a frequency nor harm to users. **Model limits.** TURNSTILE models no timers (codex-acp's 10-minute wait), not Codex's V1 protocol, and no Codex version after 0.160.1 (0.162.0 ran only live). In the cancel and interrupt scenarios the child's turn ends 'interrupted', although a root interrupt does not stop children. The proofs rest on our reading of Codex and codex-acp source, not on a mechanized model, and the Claude assumptions were checked against the adapter and six live runs; two had a CLI resume, which the stub omits. **Contract.** Both adapters and their tests aggregate child-turn paths, which the AIR documentation leaves unassigned. The invariant is our reading and has not yet been raised with the maintainers. **Fixes.** All patches are experimental. Besides the gaps of Sec. III, the Codex start-time and `rootTurnId` patches drop child turns started before their spawn item. The dispatch-time patch can still misattribute a child turn that has no dispatch item or was never observed (for example, a sleeping child woken by mail), which we do not model. **Independence.** Violations within an adapter share one mechanism, so counts measure the failing region, not independent bugs. Both adapters are main-branch snapshots. All reviews of the tool, fixes, and proofs were internal.

## VIII. Related Work

Mishra and Sharad [10] bind delegation context at execution time because trace reconstructions grow ambiguous with concurrency; we claim neither that idea nor systematic exploration as new. Stop Means Stop [11], Lingering Authority [12], and Li [13] study whether an effect may happen; we ask whom it is reported against. AgentThread [14] model-checks clauses of five agent protocols, ACP among them, in TLA+ for security properties; TURNSTILE explores the unmodified implementation. HarnessTester [15] generates harness unit tests, not event orders. Bug studies of agent frameworks [16]–[18] and coding agents [19] mine issue trackers, and MAST [20] classifies failures visible in multi-agent conversations, where a misattributed report does not show. Like CHESS [3] (threads) and MoDist [21] (messages), TURNSTILE controls delivery to an unmodified system.

## IX. Discussion and Conclusion

Arrival time is a wrong proxy for the request that caused delegated work: per-request bookkeeping cannot track work that outlives the request. Start-time binding fixes our Claude scenarios, not the Codex ones, but failed live on a Claude Code resume; dispatch-time binding removes every misattribution in our Codex scenarios at a cost in recall. On the typed Codex 0.160.1 wire no adapter meeting our conditions is exact in every legal execution, while origin carried on the event stream suffices (Prop. 1). This argues for lineage on every delegated event. By source reading, Codex 0.162.0's `rootTurnId` provides it under the conditions of Sec. III; our patch reading it is not yet explored with TURNSTILE. A request id on delegated ACP events and an "unassigned" report bucket would spare adapters the choice between guessing and dropping. Our tool, traces, and patches are not yet public.


## Acknowledgment

AI assistants (Anthropic Claude, OpenAI Codex) contributed to every section: they read source code, wrote tests, patches, stubs, scenarios, and figure scripts, ran experiments, and drafted and edited the text, proofs, source screen, and internal reviews. The authors take full responsibility for the paper.


## References


[1] Agent Client Protocol contributors, "Agent Client Protocol: Prompt turn and cancellation," https://github.com/agentclientprotocol/agent-client-protocol, docs/protocol/v1/prompt-turn.mdx, Oct. 2026.

[2] Agent Client Protocol contributors, "codex-acp: AIR extensions, agent file change report," https://github.com/agentclientprotocol/codex-acp, docs/air-extensions.md, commit 082be156.

[3] M. Musuvathi, S. Qadeer, T. Ball, G. Basler, P. A. Nainar, and I. Neamtiu, "Finding and reproducing heisenbugs in concurrent programs," in *Proc. 8th USENIX OSDI*, 2008, pp. 267–280.

[4] V. Briliantov and B. Brandt, "Subagent sessions (RFD)," https://agentclientprotocol.com/rfds/subagents, commit bfeaad0c.

[5] OpenAI, "Codex: app-server and core source, release rust-v0.160.1," https://github.com/openai/codex, commit d27764b8.

[6] N. A. Lynch, "A hundred impossibility proofs for distributed computing," in *Proc. 8th ACM PODC*, 1989, pp. 1–28.

[7] OpenAI, "Codex pull request #51415: Expose and persist turn lineage across the app server," https://github.com/openai/codex/pull/51415, Oct. 2026.

[8] Agent Client Protocol contributors, "codex-acp source: CodexSubagentEventRouter," https://github.com/agentclientprotocol/codex-acp, commit 082be156.

[9] Agent Client Protocol contributors, "claude-agent-acp pull request #870: fix: hold a turn open while its background subagents are still live," https://github.com/agentclientprotocol/claude-agent-acp/pull/870, Jul. 2026.

[10] A. Mishra and K. Sharad, "Observability for delegated execution in agentic AI systems," arXiv:2606.09692, 2026.

[11] S. Khan, "Stop means stop: Measuring and repairing the enforcement gap in agent-framework control primitives," arXiv:2607.14166, 2026.

[12] I. Santos-Grueiro, "Lingering authority: Revocable resource-and-effect capabilities for coding agents," arXiv:2606.22504, 2026.

[13] J. Li, "Where does exactly-once live? Model, harness, and tool-contract effects on duplicate side effects in LLM agents," arXiv:2609.29095, 2026.

[14] S. Zheng, Q. Zhang, Z. Zhang, H. Li, and C. Hauser, "Formal security analysis of agent protocol composition," arXiv:2606.28690, 2026.

[15] Y. Xiong, J. Ge, Z. Chen, and Y. Lou, "Complex agents, shallow tests: Demystifying and enhancing test adequacy of agent harness in the wild," arXiv:2610.04921, 2026.

[16] X. Zhu, Y. Mu, X. Zhang *et al.*, "Where agent frameworks fall short: Examining functional challenges and usability concerns," arXiv:2602.21806, 2026.

[17] X. Zhang, H. Zhang, and S. H. Tan, "Understanding bugs in modern agentic frameworks: A study of symptoms, root causes, and triggering conditions," arXiv:2604.08906, 2026.

[18] J. Chen, S. Chen, H. Zhu, J. Cao, J. Shen, and S.-C. Cheung, "Understanding agent-reactive bugs at the model-harness boundary: An empirical study of LLM agent issue reports," arXiv:2607.15684, 2026.

[19] R. Zhang, W. Dai, H. V. Pham, G. Uddin, J. Yang, and S. Wang, "Engineering pitfalls in AI coding tools: An empirical study of bugs in Claude Code, Codex, and Gemini CLI," in *Proc. 34th ACM FSE*, 2026, pp. 393–403.

[20] M. Cemri, M. Z. Pan, S. Yang *et al.*, "Why do multi-agent LLM systems fail?" arXiv:2503.13657, 2025.

[21] J. Yang, T. Chen, M. Wu *et al.*, "MODIST: Transparent model checking of unmodified distributed systems," in *Proc. 6th USENIX NSDI*, 2009, pp. 213–228.